Общее описание

- Лабораторные работы - это поэтапная работа над одним проектом. **В связи с этим, перед выполнением работ, рекомендуется предварительно ознакомиться с текстом всех работ.**
- Лабораторные работы выполняются и защищаются индивидуально.
- За лаб. работу может быть выставлено неполное количество баллов в случае наличия ошибок или замечаний. Невыполнение требования полностью рассматривается как невыполнение лаб. работы, либо понижение максимального балла.

Требования к коду

1. Обязательное соблюдение стиля кода: отсутствие транслитераций в названиях переменных, функций, классов и.т.д; все идентификаторы должны быть “говорящими, то есть по названию должен быть понятен смысл идентификатора; вложенность блоков должна быть обозначена табуляцией; каждому .cpp файлу (кроме файла с ф-цией main) должен соответствовать .h файл; в каждом .h файле объявлен только 1 класс.
   Гайд по стилю для примера https://google.github.io/styleguide/cppguide.html
2. Весь проект должен быть написан в ООП стиле.
   Все функции кроме main должны быть методами классов.
   Количество static методов должно быть минимально, а их использование четко обосновано.
   В функции main должна быть только инициализация необходимых объектов и запуск логики (для первой лаб. работы можно в main добавить демонстрационный код).
3. Весь код должен быть написан на чистом C++ с использованием стандартной библиотеки. Сторонние библиотеки можно использовать только для написания GUI или CLI. *Исключение, библиотеки по работе с файлами, например JSON, но не библиотеки, которые реализуют больше логики чем работа с файлами (например, сериализация/десериализация)*
4. Все классы должны реализованы так, чтобы гарантировалось сохранение инварианта.
5. Запрещается дублирование кода, в том числе смысловое.
6. Не должно быть “божественных” классов.
7. Запрещен сгенерированный код.
8. При оценке кода учитывается его понятность и читаемость, его масштабируемость для нового функционала, а также то, насколько легко вносить изменения в существующую логику.

Требования к отчету

1. Отчет должен быть оформлен согласно шаблону с ЛЭТИ
2. В отчете должны быть следующие разделы:
    a. Описание архитектуры. Должна быть представлена UML-диаграмма классов (отображать только классы из лаб. работы, а также классы из предыдущих лаб. работ, которые связаны с текущей работой). Должно быть представлено текстовое описание архитектуры, обоснование архитектурных решений, используемые паттерны проектирования. Если вносились изменения в код предыдущих лаб. работ, то нужно также описать их в данном разделе, и обосновать необходимость изменений.
    b. Описание классов. Описать назначение классов, обоснование наличие методов класса. Просто перечислить список методов класса не является описанием. Не обязательно описывать каждый метод.
    c. Тестирование. Продемонстрировать работоспособность классов и программы в целом.
    d. Заключение.
3. Для каждой лаб. работы пишется отдельный отчет, описывающий ровно то, что было сделано в рамках лабораторной работы.
4. Сгенерированные отчеты не принимаются, как и вся лаб. работа в таком случае.

Сдача лаб. работ

1. При выполнении работы студент обязан выполнить все основные требования. Дополнительные требования выполняются по желанию для возможности получить больше баллов. Какие именно дополнительные требования выполнять, студент также определяем самостоятельно.
2. После первой сдачи лаб. работы, **нельзя доделать дополнительные требования** для получения большего количества баллов, они фиксируются в момент первой попытки защитить лаб. работу. **Невыполнение какого-либо основного требования**, расценивается, как невыполнение лаб. работы, и лаб. работа не принимается (не ставится никакой балл).
3. Сдача лаб. работы до дедлайна позволяет получить максимальный балл. Сдача в течение трех недель после дедлайна позволяет получить максимум половину баллов (3 балла за основные требования и по 1 баллу за дополнительные. Сдача после трех недель после дедлайна дает минимальный балл (1 балл за основные требования, 0 баллов за дополнительные).
4. Если работа сдана до дедлайна, то **при защите следующей лаб. работы** можно показать исправления и добрать до **`максимального балла-1`** баллов за лаб. работу (максимум 11 баллов, если выполнялись все доп. требования). Для последней лаб. работы исправления показываются отдельно.
   Исправления можно показать только 1 раз.
5. За одно занятие лаб. работу можно показать только **один** раз.

Запись на защиту проводится до 12:00 среды (гр. 5381, 5341-44)
Таблица с дедлайнами и баллами
Даты могут быть скорректированы в течение семестра

Л/Р	**Дедлайн**	**Половина баллов**	**Минимум**
1	~~10.10~~ 17.10	~~31.10~~ 07.11	после ~~31.10~~ 07.11
2	14.11	05.12	после 05.12
3	19.12	09.01 (2027)	после 09.01 (2027)

Оценка за семестр

- Оценку за семестр можно получить на основании набранных баллов за работы, если сданы все 3 лаб. работы.
- Можно воспользоваться возможностью поменять оценку сдав экзамен. В таком случае баллы “сгорают”.
- Для допуска к экзамену (получению оценки) необходимо сдать первые 3 работ. Если сдано меньше 3х лаб. работ, то ставится оценка **Неудовлетворительно**.
- Если сданы 3 работы, но не набрано достаточное кол-во баллов, то допуск к экзамену есть и он сдается в обязательном порядке.
- **Если при проверке лаб. работ есть подозрения на использование нейросетей для генерации кода и отчета, то вне зависимости от набранных баллов, экзамен сдается в любом случае.**

Соотношение оценки и кол-ва баллов (включительно):

Оценка	**Баллы**
Отлично	31
Хорошо	23
Удовлетворительно	15

Лабораторные работы

В рамках лабораторных работ необходимо реализовать игру с пошаговым перемещением игрока и сражением с врагами.

Лабораторная работа №1

Основные требования (6 баллов):

- Реализовать класс робота. У робота должны быть характеристики: кол-во здоровья и максимальное здоровье, показатель атаки, количество энергии и максимальная энергия, количество текущего опыта и кол-во опыта необходимо для повышения ранга. Для характеристик добавить методы, позволяющие изменять значения с контролем допустимых значений.
  Добавить метод для взаимодействия с другими роботами. Если робот вражеский, то наносится атака, если дружественный, происходит лечение определенного количества жизней.
  Предусмотреть повышение ранга (увеличение характеристик) при получении необходимого количества опыта.
- Реализовать класс игрового поля, состоящего из клеток. Размер игрового поля задается аргументами конструктора, должен быть контроль допустимого размера (не меньше и не больше определенных размеров). Поле должно хранить информацию о конфигурации поля, какие клетки проходимы, и какие нет. На непроходимые клетки не могут становиться роботы. Рекомендуется добавить класс клетки для хранения информации.
- Создать систему перемещения роботов по игровому полю. Роботы не могут наступить на непроходимые клетку. При попытке перейти на клетку, занятую другим роботом, должно происходит взаимодействие.
- Создать систему игрового цикла с пошаговыми ходами. Каждый ход заключается в том, что сначала игрок выполняет действия своего робота (игрок контролирует ровно одного робота), затем движения выполняют остальные роботы под контролем компьютера. В конце каждого хода, каждый робот восстанавливает часть энергии. После завершения хода, должен наступать следующий ход. (на данном этапе, прохождением уровня можно считать уничтожение всех вражеских роботов).
- Разработать простейшую систему визуализации поля, на которой отображаются проходимые/непроходимые клетки, расположение робота игрока и вражеских роботов. Классы робота/поля не должны выводить ничего, и знать какую-либо информацию о том, как должны отрисовываться. Система визуализации должна собирать информацию и визуализировать текущую игровую ситуацию.

Дополнительные требования (2 балла за каждое требование):

- У робота добавить характеристику скорости. Значение скорости определяет как далеко работ может пройти за один ход. Добавить степень сложности проходимости клеток, чем выше степень проходимости, тем сильнее она уменьшает расстояние, которое можно пройти далее.
- Добавить систему видимости. Характеристика робота, определяющая, сколько клеток вокруг себя он видит. Клетки вне видимости скрыты от игрока, и он не знает, что на них находится (в том числе проходимость клеток). Если игрок

увидел расположение непроходимых клеток или завод, то они становятся известными, и отображаются, даже если вне видимости робота игрока.
- Создать класс завода роботов. Завод размещается на игровом поле и занимает площадь 2 на 2 клетки. Раз в несколько ходов (задается через аргумент конструктора) завод создаёт врага рядом с собой.

Лабораторная работа №2

Основные требования (6 баллов):

- Создать класс-интерфейс способности робота, которые он может применять. **Класс-интерфейса не должен ничего хранить, содержать только публичные чисто виртуальные методы.**
  Сделать реализацию интерфейса, создав классы следующих способностей:
  - Удар вокруг - нанести урон всем роботам в радиусе 1-й клетки.
  - Дальний удар - нанести урон одному выбранному роботу в радиусе 3-х клеток.
  - Лечение - восстановить фиксированное количество здоровья роботу, использующему способность.
  - Телепортация - переместиться на выбранную клетку в радиусе 5-и клеток, если она является проходимой и не занята другим роботом.

  Каждая способность требует энергию для использования. В начале игры, игрок выбирает начальную способность, которая будет у него доступна сразу. Вражеские роботы получают случайно способность Лечение или Телепортация. За один ход, робот может применить способность сколько угодно раз, пока есть энергия для их использования.
- Модифицировать повышение ранга. У робота игрока при повышении ранга дополнительно должна быть возможность, выбрать еще одну способность (если не все получены), либо улучшить, которая имеется. Улучшения: Удар вокруг - увеличение радиуса, Дальний удар - увеличение урона, Лечение - увеличение лечения, Телепортация - снижение количества необходимой энергии.
- Создать класс-интерфейс статуса. Статусы применяются к роботам, поэтому нужно добавить методы для работы и обработки статусов.
  Реализовать следующие статусы:
  - Замедление - запрещает передвижение на один ход, статус пропадает в конце хода. При получении нескольких замедлений, оно длится несколько ходов.
  - Перегрузка - запрещает использование способности, статус пропадает при попытке использовать способность (способность не активируется). При получении нескольких перегрузок, должно срабатывать как одна перегрузка.
  - Горение - наносит урон в течение нескольких ходов (количество ходов задается через аргумент конструктора статуса), статус пропадает спустя указанное количество ходов. При получении нескольких горений, продлевается длительность статуса.
  - Щит - Блокирует фиксированное количество единиц урона, статус спадает при получении урона больше значения щита (разница в уроне в таком случае наносится по здоровью). При получении нескольких щитов, увеличивается количество блокируемого урона.

- Добавить к клеткам игрового поля возможность накладывать статусы. При генерации игрового поля, клетка может получить от 0 до 2 статусов, которые накладываются на робота, когда он наступает на клетку. Чем больше статусов на клетке, там меньше вероятность появления таких клеток. После того, как робот наступил на клетку, убирается один случайный статус, который она может наложить.

Дополнительные требования (2 балла за каждое требование):

- Модифицировать способности следующим образом:
    - Удар вокруг - дополнительно накладывает статус Перегрузка.
    - Дальний удар - дополнительно накладывает статус Горение.
    - Лечение - дополнительно накладывает статус Щит при попытке вылечить здоровье больше максимального. Размер щита равен значению, на которое лечение превышает максимальное.
- Добавить систему инвентаря. Инвентарь есть только у робота игрока. Добавить предметы восстанавливающие здоровье, восстанавливающие энергию, временно повышающие атаку. Для получения предметов добавить статус получения предмета, который срабатывает только на робота игрока.
- Модифицировать систему визуализации, добавив режим переключения отображения. Стандартный, подсветка врагов, подсветка непроходимых клеток, подсветка клеток, накладывающих статусы (для CLI подсветка - выделение определенным цветом).

Лабораторная работа №3

Основные требования (6 баллов):

- Создать шаблонный класс игры. Данный класс отвечает за игровой цикл. Инициализация игры, переход между уровнями при завершении, завершение игры (при выигрыше или проигрыше).
  В качестве параметра шаблона должен принимать имя класса, который отвечает за генерацию уровней и их количество. Должны быть следующие варианты запуска игры:
    - Фиксированные уровни. Уровни не генерируются, а считываются из файла.
    - Случайные уровни. Уровень генерируется случайно. Каждый последующий уровень должен быть сложнее (вражеские ранги сразу высокие, больше клеток со статусами). Количество уровней фиксировано в параметре шаблона для данной стратегии.
    - Бесконечный режим. Уровень генерируется случайно. Каждый последующий уровень должен быть сложнее. Количество уровней неограниченно, игра продолжается до поражения игрока.

  При переходе на следующий уровень, робот игрока сохраняет уровень, способности и их улучшения, значения опыта и здоровья. Энергия должна сбрасываться, а все статусы сниматься.
  Также у игрока должна быть возможность выйти из игры в меню, и выбрать новую игру или загрузить существующую.
- Организовать системы ввода команд и визуализации. Системы ввода и визуализации должны быть независимы друг от друга. Ввод команд и визуализация не должны происходить внутри класса игры или в каких-либо

игровых сущностях. Только передача команд. Система ввода команд должна считывать нажатие клавиши пользователя и преобразовывать ее в игровую команду. Конфигурация управления (то какая клавиша за что отвечает) должно задаваться через файл. Если файл некорректный (не на все команды назначена клавиша, одна клавиша отвечает за несколько команд, на одну команду разные клавиши), то управление должно устанавливаться по-умолчанию с оповещения пользователя.

- Разработать систему сохранения/загрузки. Данная система не должно быть встроена в класс игры. Сохранение должно записывать состояние игры в файл, сохраниться можно в любой момент игры, когда игрок может выполнять действия. Загрузка должна восстанавливать состояние игры из файла, загрузиться можно при запуске программы (выбрать новую игру или загрузиться), а также во время игры, когда игрок может совершать действия. При загрузке должна быть проверка валидности данных, если загрузиться невозможно, то программа должна быть в том состоянии, которое было до попытки загрузиться.

Дополнительные требования (2 балла за каждое требование):

- Добавить выбор сложности игры. Сложность игры влияет на силу вражеских роботов, а также случайно добавляет модификаторы из списка:
    - Можно использовать только 1 способность 1 раз за ход.
    - Каждый ход к игроку применяется случайный статус.
    - Статусы не снимаются при переходе с уровня на уровень.
    - Нельзя улучшать способности при повышении ранга.
- Расширить систему визуализации таким образом, чтобы через параметр шаблона можно было выбирать стандартную визуализацию или визуализацию настраиваемую через файл (в файле задаются символы и цвет символов для отображения сущностей в разных режимах).
- Добавить систему логирования событий в программе. Логирование должно записывать в файл то, что происходит в программе. Должны быть следующие уровни логирования:
    - Информация о программе - попытка загрузки, сохранение, старт новой игры, выход в меню, и т.д..
    - Информация об игре - кто какие ходы выполняет, куда перемещается, какие способности и как использует, повышение ранга, переход на следующий уровень.
    - Ошибки программы - такие как невозможность загрузиться и причины (отсутствие файла или некорректный файл) и т.д.
    - Ошибки в игре - попытка переместиться на непроходимую клетку, нелегитимная цель для способности, попыкатка выйти за границу поля и т.д.

  Пользователь должен иметь возможность выбрать любую комбинацию уровней логирования, в том числе никак не логировать. Через параметр шаблона должно задавать, записывать ли все логи в один файл, или записывать информацию и ошибки в разные файлы.